Polynomials and FFT solutions

30-E01 Degree bounds and aliasing

The product has degree at most 5, hence degree bound 6. At least six distinct values guarantee uniqueness. The smallest positive power of two at least 6 is 8. Pad both inputs to 8 rather than retaining their original lengths.

The two-point DFT of $1 + x$, at 1 and -1, is `[2,0]`. Squaring pointwise gives `[4,0]`. The inverse is $[(4 + 0)/2, (4 - 0)/2] = [2, 2]$. The ordinary product is $1 + 2x + x^2$, but modulo $x^2 - 1$, $x^2 \equiv 1$, so the constant and quadratic coefficients combine to give $2 + 2x$. The DFT correctly solved a cyclic convolution problem.

30-E02 Every FFT intermediate value

Original indices 0 and 2 hold `[2,7]`, whose DFT is `[9,-5]`. Original indices 1 and 3 hold `[5,-1]`, whose DFT is `[4,6]`. After bit reversal the physical array is `[2,7,5,-1]`.

Stage 1 has $m = 2$. Physical positions `(0,1)` use $w = 1$ and produce 9,-5. Positions `(2,3)` use $w = 1$ and produce 4,6. The resulting array is `[9,-5,4,6]`. These pairs contain original indices `(0,2)` and `(1,3)`, respectively.

Stage 2 has $m = 4$. Positions `(0,2)` use $w = 1, u = 9, t = 4$, producing $A_0 = 13, A_2 = 5$. Positions `(1,3)` use $w = i, u = -5, t = 6i$, producing $A_1 = -5 + 6i, A_3 = -5 - 6i$. Output order is `[13,-5+6i,5,-5-6i]`. The checks $y_0 = 13$ and $y_2 = A(-1) = 2 - 5 + 7 + 1 = 5$ agree. Stage 2 crosses half-blocks rather than pairing adjacent cells again.

30-E03 Inverse sign and normalization

Let $F[k, j] = \omega^{jk}$, $G[j, k] = \omega^{-jk}/n$. Entry (j, ℓ) of GF equals $\sum_k \omega^{k(\ell-j)}/n$, which is 1 for $j = \ell$ and 0 otherwise. Thus $G = F^{-1}$. The negative exponent and normalization follow from the same identity.

In $F^2 a/n$, position j receives $\sum_\ell a_\ell \sum_k \omega^{k(j+\ell)}/n$. Only $\ell \equiv -j \pmod n$ survives. For the stated input the result is `[2,-1,7,5+i]`, whereas conjugation gives `[2,5-i,7,-1]`. Using the correct negative sign but omitting division by 4 gives `[8,20+4i,28,-4]`. Wrong signs reverse cyclic indices. Missing normalization scales every value.

Polynomials and FFT solutions (continued)

30-E04 Existence and uniqueness of interpolation

The basis $\ell_k(x) = \prod_{j \neq k}(x - x_j)/(x_k - x_j)$ satisfies $\ell_k(x_i) = \delta_{ik}$. Therefore $\sum_k y_k \ell_k$ matches every point and has degree below n. Any other matching polynomial B makes $A - B$ a polynomial of degree below n with n distinct roots. Repeated use of the factor theorem proves that a nonzero degree-d polynomial has at most d roots. Hence $A - B = 0$.

For the points 0 and 1, $A_t(x) = 1 + 2x + tx(x - 1)$, with arbitrary t, always gives 1 and 3 and has degree below 3. Different t give different quadratic coefficients, so there are infinitely many solutions. Requiring degree below 2 removes that term and leaves the unique polynomial $1 + 2x$. Both the degree bound and enough distinct points are necessary.

30-E05 Three transforms for convolution

Zero padding gives `[1,2,0,0]` and `[1,-1,0,0]`. At $1, i, -1, -i$, their transforms are `[3,1+2i,-1,1-2i]` and `[0,1-i,2,1+i]`. Their pointwise product is `[0,3+i,-2,3-i]`, since $(1 + 2i)(1 - i) = 3 + i$ and $(1 - 2i)(1 + i) = 3 - i$.

The negative-exponent DFT is `[4,4,-8,0]`. Divide by 4 to get `[1,1,-2,0]`, or $1 + x - 2x^2$. Since $4 \geq 2 + 2 - 1$, no coefficient wraps around.

An eight-point FFT has $\log_2 8 = 3$ stages and four butterflies per stage, giving 12 butterflies. Serial execution does all 12. A bounded-fan-in circuit with all butterflies in a stage parallel, and with twiddles and wires ready, has depth three stages. Finite processors and communication have additional costs. The three transforms each cost $\Theta(N \log N)$, and pointwise multiplication costs $\Theta(N)$. The total remains $\Theta(N \log N)$, while W/P bounds time with P processors.

30-E06 In-place overwrites and the stage invariant

The correct outputs are $9 + 4 = 13$ and $9 - 4 = 5$. Reading the updated upper cell would instead give $13 - 4 = 9$ below. First set `u=A[top]` and `t=w*A[bottom]`, then write `A[top]=u+t` and `A[bottom]=u-t`. Both writes use the old values.

At stage s, $m = 2^s$. Before the stage, the two halves of each length-m block hold the appropriate $m/2$-point even and odd transforms. After combining them, the block holds the complete m-point transform. Singletons prove the base case, and induction reaches $m = n$. Physical separation is $m/2$, and the twiddle uses m, not the stage number s.

Polynomials and FFT solutions (continued)

Length 6 is not a power of two. Halving gives length 3, which cannot be split into two equal integer-sized subproblems at the next level. Other radices or mixed-radix algorithms can handle it. For ordinary polynomial convolution, padding to a sufficiently large power of two is valid. Padding a six-point signal to eight points instead changes the evaluation points and computes an eight-point DFT, not the original six-point DFT.

30-E07 NTT conditions and signed reconstruction

We have $13^2 = 169 \equiv 16 = -1 \pmod{17}$, so $13^4 \equiv 1$. Neither 13 nor its square is 1, proving order exactly 4. Also $4 \cdot 13 = 52 \equiv 1$, so $4^{-1} = 13$, and $\omega^{-1} = 4$.

The NTTs are `[3,10,16,9]` and `[0,5,2,14]`. Their product modulo 17 is `[0,50 mod 17,32 mod 17,126 mod 17]=[0,16,15,7]`. Inverting with ω^{-1} and 4^{-1} gives `[1,1,15,0]`.

The original product coefficients have absolute value at most $\min(2,2)2^2 = 8$. Every modulo-17 residue has exactly one integer representative in $[-8, 8]$, and 15 represents -2. With a bound of 20, the same residue 15 may mean -19, -2, or 15. Unique recovery then requires a larger prime with a suitable root, or several moduli with CRT and a sufficiently large total modulus. Exact field arithmetic alone does not guarantee the original integers. Invertibility of N, the root's order, and the signed magnitude bound are separate requirements.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT solutions (continued)

Verification code appendix: complex_dft_roundtrip.py

```
"""Naive DFT and inverse DFT verify the FFT contract."""

import cmath

def dft(values, inverse=False):
    n = len(values)
    sign = -1 if inverse else 1
    result = []
    for k in range(n):
        total = 0j
        for j, value in enumerate(values):
            total += value * cmath.exp(sign * 2j * cmath.pi * j * k / n)
        result.append(total / n if inverse else total)
    return result


source = [1.0, 2.0, 0.0, -1.0]
expected = [2, 1+3j, 0, 1-3j]
assert max(abs(a-b) for a,b in zip(dft(source), expected)) < 1e-9
roundtrip = dft(dft(source), inverse=True)
assert max(abs(a - b) for a, b in zip(source, roundtrip)) < 1e-9
print([round(value.real, 9) for value in roundtrip])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT solutions (continued)

Verification code appendix: fft_depth_checks.py

```
"""Positive-exponent FFT, independently compared to direct evaluation/convolution."""
from cmath import exp
from math import pi
from itertools import product
import random


def bitreverse(index,bits):
    result=0
    for _ in range(bits):result=(result<<1)|(index&1);index>>=1
    return result


def iterative(values,inverse=False):
    n=len(values)
    if n==0 or n&(n-1):raise ValueError('radix2 length must be a positive power of2')
    bits=n.bit_length()-1;a=[complex(values[bitreverse(i,bits)]) for i in range(n)];stages=
[a[:]];wires=[]
    m=2
    while m<=n:
        wm=exp((-1 if inverse else 1)*2j*pi/m)
        for base in range(0,n,m):
            w=1
            for j in range(m//2):
                top,bottom=base+j,base+j+m//2;u=a[top];v=w*a[bottom]
                wires.append((m,top,bottom,w,u,v))
                a[top]=u+v;a[bottom]=u-v;w*=wm
        stages.append(a[:]);m*=2
    if inverse:a=[v/n for v in a]
    return a,stages,wires


def recursive(values,inverse=False):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT solutions (continued)

```
    n=len(values)
    if n==0 or n&(n-1):raise ValueError('radix2 length must be a positive power of2')
    def transform(a):
        n=len(a)
        if n==1:return [complex(a[0])]
        e,o=transform(a[::2]),transform(a[1::2]);out=[0j]*n
        for k in range(n//2):
            term=exp((-1 if inverse else 1)*2j*pi*k/n)*o[k]
            out[k]=e[k]+term;out[k+n//2]=e[k]-term
        return out
    answer=transform(values)
    return [v/n for v in answer] if inverse else answer


def dft(values,inverse=False):
    n=len(values);sign=-1 if inverse else 1
    return [sum(a*exp(sign*2j*pi*j*k/n) for j,a in enumerate(values))/(n if inverse else 1)
for k in range(n)]


def close(a,b):return len(a)==len(b) and all(abs(x-y)<1e-8 for x,y in zip(a,b))


def convolution(a,b):
    if not a or not b:return []
    length=len(a)+len(b)-1;n=1
    while n<length:n*=2
    ya=iterative([*a,*([0]*(n-len(a)))])[0];yb=iterative([*b,*([0]*(n-len(b)))])[0]
    return iterative([u*v for u,v in zip(ya,yb)],True)[0][:length]


def direct_convolution(a,b):
    if not a or not b:return []
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT solutions (continued)

```
    answer=[0]*(len(a)+len(b)-1)
    for i,u in enumerate(a):
        for j,v in enumerate(b):answer[i+j]+=u*v
    return answer


def ntt(values,prime,root,inverse=False):
    n=len(values)
    assert pow(root,n,prime)==1
    assert all(pow(root,k,prime)!=1 for k in range(1,n))
    omega=pow(root,-1,prime) if inverse else root
    answer=[sum(v*pow(omega,j*k,prime) for j,v in enumerate(values))%prime for k in range(n)]
    return [(v*pow(n,-1,prime))%prime for v in answer] if inverse else answer


def main():
    transforms=0;products=0
    for n,alphabet in [(1,range(-2,3)),(2,range(-2,3)),(4,range(-2,3)),(8,[-1,0,1])]:
        for values in product(alphabet,repeat=n):
            expected=dft(values);actual,stages,wires=iterative(values)
            assert close(actual,expected) and close(recursive(values),expected)
            assert close(iterative(actual,True)[0],values) and close(dft(actual,True),values)
            assert len(wires)==n//2*(n.bit_length()-1)
            transforms+=1
    rng=random.Random(30)
    for n in [16,32,64]:
        for _ in range(30):
            values=[complex(rng.randrange(-9,10),rng.randrange(-9,10)) for _ in range(n)]
            assert close(iterative(values)[0],dft(values));transforms+=1
    polys=[values for n in range(5) for values in product([-1,0,1],repeat=n)]
    for a in polys:
        for b in polys:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT solutions (continued)

```
            assert close(convolution(a,b),direct_convolution(a,b));products+=1
    result,stages,wires=iterative([2,5,7,-1])
    assert close(stages[0],[2,7,5,-1]) and close(stages[1],[9,-5,4,6])
    assert [(u,v) for m,u,v,*_ in wires if m==4]==[(0,2),(1,3)]
    assert close(result,[13,-5+6j,5,-5-6j])
    assert close(iterative([1,2,0,-1])[0],[2,1+3j,0,1-3j])
    ya=dft([1,2,0,0]);yb=dft([1,-1,0,0]);combined=[a*b for a,b in zip(ya,yb)]
    assert close(ya,[3,1+2j,-1,1-2j]) and close(yb,[0,1-1j,2,1+1j])
    assert close(combined,[0,3+1j,-2,3-1j]) and close(dft(combined,True),[1,1,-2,0])
    # Underpadding produces cyclic convolution, not the ordinary product.
    shorta=dft([1,1]);short=[v*v for v in shorta]
    assert close(dft(short,True),[2,2])
    na=ntt([1,2,0,0],17,13);nb=ntt([1,-1,0,0],17,13)
    assert na==[3,10,16,9] and nb==[0,5,2,14]
    assert [(a*b)%17 for a,b in zip(na,nb)]==[0,16,15,7]
    assert ntt([0,16,15,7],17,13,True)==[1,1,15,0]
    for bad in [[],[1,2,3]]:
        try:iterative(bad)
        except ValueError:pass
        else:raise AssertionError('invalid radix2 input accepted')
    # Wrong positive sign in an inverse gives circular index reversal, not a general
conjugation.
    asymmetric=[2,5+1j,7,-1]
    assert close([v/4 for v in iterative(iterative(asymmetric)[0])[0]],
                 [asymmetric[0],asymmetric[3],asymmetric[2],asymmetric[1]])
    print(f'PASS FFT_direct_evaluation_cases={transforms} polynomial_convolution_cases=
{products} '
          'recursive_and_iterative_and_inverse=True distinct_stage_values_and_wires=True '
          'zero_padding_alias=2,2 NTT_mod17_exact=1,1,15,0
inverse_wrong_sign=index_reversal')


if __name__=='__main__':main()
```

Polynomials and FFT solutions (continued)

Verification code appendix: fft_trace.py

```
"""Verify the required radix-2 FFT stages and inverse round trip."""

import cmath


def bit_reverse_two_bits(value):
    return ((value & 1) << 1) | ((value >> 1) & 1)


def staged_fft(values, inverse=False):
    n = len(values)
    assert n == 4
    stages = []
    a = [values[bit_reverse_two_bits(i)] for i in range(n)]
    origins = [bit_reverse_two_bits(i) for i in range(n)]
    for half in (1, 2):
        block = 2 * half
        root = cmath.exp((2j if not inverse else -2j) * cmath.pi / block)
        for start in range(0, n, block):
            weight = 1
            for j in range(half):
                u = a[start + j]
                t = weight * a[start + j + half]
                a[start + j] = u + t
                a[start + j + half] = u - t
                weight *= root
        stages.append(tuple(a))
    if inverse:
        a = [item / n for item in a]
    return a, stages, origins


def close_pair(actual, expected):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT solutions (continued)

```
    return all(abs(x - y) < 1e-9 for x, y in zip(actual, expected))


def direct_dft(values, inverse=False):
    n = len(values)
    sign = -1 if inverse else 1
    return [
        sum(values[j] * cmath.exp(sign * 2j * cmath.pi * j * k / n) for j in range(n)) / (n
if inverse else 1)
        for k in range(n)
    ]


def run():
    source = [1, 2, 0, -1]
    output, stages, origins = staged_fft(source)
    assert origins == [0, 2, 1, 3]
    assert close_pair(stages[0], [1, 1, 1, 3])
    expected = [2, 1 + 3j, 0, 1 - 3j]
    assert close_pair(output, expected)
    assert close_pair(output, direct_dft(source))
    assert close_pair(direct_dft(direct_dft(source), inverse=True), source)
    recovered, inverse_stages, _ = staged_fft(output, inverse=True)
    assert close_pair(recovered, source)
    assert len(stages) == 2 and len(inverse_stages) == 2
    print("PASS fft origins=0,2,1,3 stage1=1,1,1,3 output=2,1+3i,0,1-3i roundtrip=yes")
    return output


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study